

Report conversation

I am working with the following rust code for the RcDom struct. It is in a library I cannot modify:

```
/// The DOM itself; the result of parsing.
pub struct RcDom {
    /// The `Document` itself.
    pub document: Handle,

    /// Errors that occurred during parsing
    pub errors: Vec<Cow<'static, str>>,

    /// The document's quirks mode.
    pub quirks_mode: QuirksMode,
}

impl TreeSink for RcDom {
    type Output = Self;
    fn finish(self) -> Self {
        self
    }

    type Handle = Handle;

    fn parse_error(&mut self, msg: Cow<'static, str>) {
        self.errors.push(msg);
    }

    fn get_document(&mut self) -> Handle {
        self.document.clone()
    }

    fn get_template_contents(&mut self, target: NodeData::Element {
        ref template_contents,
        ..
    }) = target.data
    {

```

```

        template_contents.borrow().as_ref()
    } else {
        panic!("not a template element!")
    }
}

fn set_quirks_mode(&mut self, mode: QuirksMode) {
    self.quirks_mode = mode;
}

fn same_node(&self, x: &Handle, y: &Handle) -> bool {
    Rc::ptr_eq(x, y)
}

fn elem_name<'a>(&self, target: &'a Handle) -> Option<'a str> {
    return match target.data {
        NodeData::Element { ref name, .. } => Some(name.as_ref()),
        _ => panic!("not an element!"),
    };
}

fn create_element(
    &mut self,
    name: QualName,
    attrs: Vec<Attribute>,
    flags: ElementFlags,
) -> Handle {
    Node::new(NodeData::Element {
        name,
        attrs: RefCell::new(attrs),
        template_contents: RefCell::new(
            Some(Node::new(NodeData::Document {
                ..
            })
        ),
        mathml_annotation_xml_integrations: Vec::new(),
    })
}

fn create_comment(&mut self, text: String) -> Handle {
    Node::new(NodeData::Document {
        ..
    })
}

```

```

Node::new(NodeData::Comment { conten

}

fn create_pi(&mut self, target: StrTendi
Node::new(NodeData::ProcessingInstru
target,
contents: data,
}))
}

fn append(&mut self, parent: &Handle, ch
// Append to an existing Text node :
match child {
NodeOrText::AppendText(ref text)
Some(h) => {
if append_to_existing_te
return;
}
},
_ => (),
},
_ => (),
}

append(
&parent,
match child {
NodeOrText::AppendText(text)
contents: RefCell::new(
)},
NodeOrText::AppendNode(node)
},
);
}

fn append_before_sibling(&mut self, sib
let (parent, i) = get_parent_and_in
.expect("append_before_sibling c

let child = match (child, i) {

```

```

// No previous node.
(NodeOrText::AppendText(text), (
    contents: RefCell::new(text)
}),

// Look for a text node before
(NodeOrText::AppendText(text), {
    let children = parent.children;
    let prev = &children[i - 1];
    if append_to_existing_text(prev) {
        return;
    }
    Node::new(NodeData::Text {
        contents: RefCell::new(text)
    })
}),

// The tree builder promises we
// the insertion point.

// Any other kind of node.
(NodeOrText::AppendNode(node), _
);

remove_from_parent(&child);

child.parent.set(Some(Rc::downgrade(&parent)));
parent.children.borrow_mut().insert(i, child);
}

fn append_based_on_parent_node(
    &mut self,
    element: &Self::Handle,
    prev_element: &Self::Handle,
    child: NodeOrText<Self::Handle>,
) {
    let parent = element.parent.take();
    let has_parent = parent.is_some();
    element.parent.set(parent);

```

```

        if has_parent {
            self.append_before_sibling(element)
        } else {
            self.append(prev_element, child)
        }
    }

fn append_doctype_to_document(
    &mut self,
    name: StrTendril,
    public_id: StrTendril,
    system_id: StrTendril,
) {
    append(
        &self.document,
        Node::new(NodeData::Doctype {
            name,
            public_id,
            system_id,
        }),
    );
}

fn add_attrs_if_missing(&mut self, target: &NodeData) {
    let mut existing = if let NodeData::Element { attrs, .. } = target {
        attrs.borrow_mut()
    } else {
        panic!("not an element")
    };

    let existing_names = existing
        .iter()
        .map(|e| e.name.clone())
        .collect:::<HashSet<_>>();
    existing.extend(
        attrs
            .into_iter()
            .filter(|attr| !existing_names.contains(attr.name))
    );
}

```

```

fn remove_from_parent(&mut self, target
    remove_from_parent(&target);
}

fn reparent_children(&mut self, node: &
    let mut children = node.children.borrow_mut();
    let mut new_children = new_parent.children.borrow_mut();
    for child in children.iter() {
        let previous_parent = child.parent.as_ref().unwrap();
        assert!(Rc::ptr_eq(
            &node,
            &previous_parent.unwrap().parent.as_ref().unwrap()
        ))
    }
    new_children.extend(mem::replace(&children, Vec::new()));
}

fn is_mathml_annotation_xml_integration_target(target: &NodeData) -> bool {
    if let NodeData::Element {
        mathml_annotation_xml_integration_target: ..
    } = target.data
    {
        true
    } else {
        panic!("not an element!")
    }
}

impl Default for RcDom {
    fn default() -> RcDom {
        RcDom {
            document: Node::new(NodeData::Document {
                errors: vec![],
                quirks_mode: tree_builder::NoQuirksMode,
            }),
        }
    }
}

```

I have another library (which I cannot modify), which exposes the function `driver::parse_document`, which accepts a `TreeSink`. Currently I am calling it like so:

```
let parser = driver::parse_document(RcDom::c
```

However, I need to customize the `TreeSink` I am passing so that instead of using the default `RcDom` behavior, I save the current line number, so that whenever `parse_error()` happens, it uses that current line number.

How can I do this, with a minimal amount of code?

To customize the `TreeSink` behavior of the `RcDom` struct to include line number tracking without modifying the original libraries, you can wrap the `RcDom` in a new struct that implements `TreeSink`. This wrapper will delegate most methods to the `RcDom` instance but override the `parse_error()` method to include line number information.

Here's an approach on how to do it:

- 1. Create a Wrapper Struct:** Define a new struct that contains an instance of `RcDom` and a field to track the current line number.
- 2. Implement `TreeSink` for the Wrapper:** Implement the `TreeSink` trait for your new struct, delegating most calls to the internal `RcDom` instance. For `parse_error()`, augment the error message with the current line number.
- 3. Override `parse_error()`:** In `parse_error()`, append or prepend the line number information to the error message before passing it to `RcDom`'s `parse_error()`.

Here's an example implementation:

rust

 Copy code

```
use html5ever::tendril::StrTendril;
use html5ever::tree_builder::{TreeSink, NodeOrText, QuirksMode, ElementFlag
use std::borrow::Cow;
use std::rc::Rc;
```

```

// Assume `RcDom` and other necessary types are imported correctly

/// A wrapper around `RcDom` that tracks the current line number.
pub struct RcDomWithLine {
    rc_dom: RcDom,
    current_line: usize,
}

impl RcDomWithLine {
    pub fn new(rc_dom: RcDom) -> Self {
        RcDomWithLine {
            rc_dom,
            current_line: 1, // Starting at line 1, adjust according to you
        }
    }

    // Method to update the current line number
    pub fn set_current_line(&mut self, line: usize) {
        self.current_line = line;
    }
}

impl TreeSink for RcDomWithLine {
    type Output = RcDom; // Output type should match the original RcDom for
    type Handle = <RcDom as TreeSink>::Handle; // Reuse the handle type from

    fn finish(self) -> Self::Output {
        self.rc_dom.finish()
    }

    fn parse_error(&mut self, msg: Cow<'static, str>) {
        // Customize the error message to include the line number
        let msg_with_line = format!("Line {}: {}", self.current_line, msg);
        self.rc_dom.parse_error(Cow::Owned(msg_with_line));
    }

    // Delegate all other methods to the internal `RcDom` instance.
    // This example shows how to delegate `get_document` method.
    // Apply a similar pattern for other methods.

```



```

    fn get_document(&mut self) -> Self::Handle {
        self.rc_dom.get_document()
    }

    // Implement all other TreeSink methods, delegating to self.rc_dom
    // ...
}

// Usage
let rc_dom = RcDom::default(); // Create an RcDom instance
let mut my_dom = RcDomWithLine::new(rc_dom); // Wrap it in your custom struct

// Example of setting the current line number before parsing
my_dom.set_current_line(42);

// Now, use `my_dom` wherever you need a `TreeSink`
let parser = driver::parse_document(my_dom, create_error_opts());

```

This code provides a basic structure. You'll need to implement or delegate all required `TreeSink` methods for `RcDomWithLine`. This pattern allows you to inject additional behavior (like line number tracking) into the parsing process without modifying the original `RcDom` or the library that uses it.

There are like 20 methods. I really need to delegate all of them using that same boilerplate pattern?

For a case with numerous methods requiring delegation, manually implementing each one can indeed be quite tedious and error-prone. To streamline this process and minimize boilerplate, you can leverage Rust's advanced features such as macros or Deref coercion, although each approach has its considerations and limitations.

Using a Macro

You can define a macro to generate delegation methods automatically. This approach can reduce the amount of code you need to write but still requires you to explicitly list all

methods you want to delegate. Here's a simplified example of how you might define such a macro:

rust

 Copy code

```
macro_rules! delegate_tree_sink_methods {
    ($($method:ident),*) => {
        $(
            fn $method(&mut self, $($arg:ty),*) -> $ret:ty {
                self.rc_dom.$method($($arg),*)
            }
        )*
    };
}
```

You'd then use this macro inside your `impl TreeSink` for `RcDomWithLine` to generate the necessary methods. Note that this macro would need to be significantly more complex to handle the full variety of methods in `TreeSink`, including those with different numbers and types of parameters, return types, etc.

Using Deref Coercion

An alternative and more elegant solution might be to use Rust's `Deref` and `DerefMut` traits to automatically delegate method calls to the `RcDom` field. This way, any method call on `RcDomWithLine` that isn't explicitly defined on it but is defined on `RcDom` will be automatically forwarded to the `rc_dom` field. Here's how you might implement it:

rust

 Copy code

```
use std::ops::{Deref, DerefMut};

impl Deref for RcDomWithLine {
    type Target = RcDom;

    fn deref(&self) -> &Self::Target {
        &self.rc_dom
    }
}

impl DerefMut for RcDomWithLine {
    fn deref_mut(&mut self) -> &mut Self::Target {
```

```

        &mut self.rc_dom
    }
}

```

With `Deref` and `DerefMut` implemented, you can call methods on `RcDom` directly on an instance of `RcDomWithLine`, and Rust will automatically delegate those calls to the `rc_dom` field. This significantly reduces boilerplate and makes your wrapper almost transparent in use. However, there are a few important considerations:

- **Explicit Methods:** You still need to explicitly implement any methods where you want to change the behavior (like `parse_error` in your case).
- **Type Coercion:** While convenient, this approach may obscure the fact that `RcDomWithLine` is not actually an `RcDom`, which can be confusing in complex codebases. It might also limit your ability to add methods to `RcDomWithLine` that have the same name as methods on `RcDom` but do different things.

Between these two approaches, using `Deref` and `DerefMut` is usually cleaner and requires less repetitive code, but the best choice depends on your specific needs and the complexity of the `TreesSink` trait implementation.

What's the difference between having a `pub fn new` for `RcDomWithLine` and doing `impl Default` for `RcDomWithLine`?

The difference between having a `pub fn new` for your struct, like `RcDomWithLine`, and implementing `Default` for it, pertains to how instances of that struct are initialized and what conventions they align with in Rust's ecosystem.

```
pub fn new
```

A `pub fn new` method is a public constructor that you define to create an instance of your struct. It allows for flexibility in initialization, enabling you to require parameters, set up any necessary internal state, or run initialization code before the struct is used. The signature and parameters of a `new` function are entirely up to you, and you can have multiple `new` functions (typically with different names) if different initialization strategies are needed.

rust

 Copy code

```
pub struct RcDomWithLine {
    rc_dom: RcDom,
    current_line: usize,
}

impl RcDomWithLine {
    pub fn new(rc_dom: RcDom) -> Self {
        RcDomWithLine {
            rc_dom,
            current_line: 1, // Example default value
        }
    }
}
```

This method is explicitly called by the programmer, making it clear what dependencies are needed and allowing for immediate customization at the point of creation.

impl Default for RcDomWithLine

Implementing the `Default` trait for `RcDomWithLine` provides a way to instantiate it with a set of default values without needing to specify them explicitly. The `Default` trait is particularly useful in generic programming when you need to create objects without knowing their concrete types or when you want to provide a simple, no-argument way to get a "default" instance of your type.

rust

 Copy code

```
impl Default for RcDomWithLine {
    fn default() -> Self {
        RcDomWithLine {
            rc_dom: RcDom::default(),
            current_line: 1, // Example default value
        }
    }
}
```

The `Default` trait enables the use of `RcDomWithLine::default()` to create an instance, and it can also be used with types that require a default value, such as when

using the `#[derive(Default)]` on larger structs that contain `RcDomWithLine`.

Key Differences and When to Use Each

- **Initialization Flexibility:** `new` can take parameters and perform complex initialization logic, making it suitable for when you need to pass in configuration options or dependencies. `Default` provides a parameterless way to create an instance with some reasonable default state.
- **Conventions and Clarity:** Using `new` makes it clear to the caller that a new instance is being created and initialized, possibly with specific provided values. `Default` is a Rust convention for types that have a sensible default value, making it a good fit for types that can be initialized without any external information.
- **Generic Programming:** `Default` is useful in generic contexts where you need to instantiate types without knowing or caring about their specific initialization details.

In practice, you might often implement both for a struct: `Default` for cases where a simple, standard initialization is fine, and one or more `new` functions for cases where more customization is required upon creation.

So I'm going to do the `Deref/DerefMut` approach. How do I augment that with custom overrides of `TreeSink`'s `parse_error` method?

To use the `Deref` and `DerefMut` approach for method delegation while also overriding specific methods like `parse_error` in your `TreeSink` implementation, you will combine the automatic delegation provided by `Deref / DerefMut` with explicit method implementations in your `impl TreeSink for RcDomWithLine`. Here's how to do it:

1. **Implement `Deref` and `DerefMut`:** This will allow your `RcDomWithLine` struct to delegate method calls to `RcDom` for any method not explicitly implemented in `RcDomWithLine`.
2. **Explicitly Implement `TreeSink` Methods:** For methods like `parse_error` that you want to override with custom behavior, you will implement them directly on `RcDomWithLine`, providing your custom logic.

Here is an example implementation illustrating this approach:

Step 1: Implement Deref and DerefMut

rust

 Copy code

```
use std::ops::{Deref, DerefMut};
use std::borrow::Cow;

// Assuming RcDom and other necessary items are already imported

pub struct RcDomWithLine {
    rc_dom: RcDom, // Encapsulated RcDom instance
    current_line: usize, // Additional state for line tracking
}

impl Deref for RcDomWithLine {
    type Target = RcDom;

    fn deref(&self) -> &Self::Target {
        &self.rc_dom
    }
}

impl DerefMut for RcDomWithLine {
    fn deref_mut(&mut self) -> &mut Self::Target {
        &mut self.rc_dom
    }
}
```

Step 2: Override TreeSink Methods

rust

 Copy code

```
impl TreeSink for RcDomWithLine {
    // Override the parse_error method
    fn parse_error(&mut self, msg: Cow<'static, str>) {
        // Custom logic here, e.g., augmenting the message with the line number
        let augmented_msg = format!("Line {}: {}", self.current_line, msg);
        self.rc_dom.parse_error(Cow::Owned(augmented_msg.into()));
    }

    // No need to manually delegate other TreeSink methods thanks to Deref
```

```
// They are automatically forwarded to self.rc_dom.
}
```

Custom Initialization and Line Number Tracking

rust

 Copy code

```
impl RcDomWithLine {
    pub fn new(rc_dom: RcDom) -> Self {
        RcDomWithLine {
            rc_dom,
            current_line: 1, // Initialize with the starting line number
        }
    }

    // Method to set the current line number
    pub fn set_current_line(&mut self, line: usize) {
        self.current_line = line;
    }
}

// Usage example
let rc_dom = RcDom::default(); // Create an RcDom instance
let mut my_dom = RcDomWithLine::new(rc_dom); // Wrap it in the custom struct

// Now, you can use my_dom as a TreeSink, with custom parse_error logic,
// while other methods are delegated to RcDom.
```

This approach gives you a clean, maintainable way to extend or modify the behavior of `RcDom` without altering the original library code. It takes advantage of Rust's type system and traits to provide a flexible and powerful method of customization.

I got this error with your approach:

```
error[E0046]: not all trait items implemented, missing:
  Handle, Output, finish, get_document, elem_name,
  create_element, create_comment, create_pi,
```

```
append, append_based_on_parent_node,  
append_doctype_to_document,  
get_template_contents, same_node,  
set_quirks_mode, append_before_sibling,  
add_attrs_if_missing, remove_from_parent,  
reparent_children  
--> src/parser.rs:92:1  
|  
92 | impl TreeSink for RcDomWithLineTracking {  
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
missing Handle, Output, finish, get_document,  
elem_name, create_element, create_comment,  
create_pi, append,  
append_based_on_parent_node,  
append_doctype_to_document,  
get_template_contents, same_node,  
set_quirks_mode, append_before_sibling,  
add_attrs_if_missing, remove_from_parent,  
reparent_children in implementation
```

The error you're encountering indicates that the Rust compiler expects all trait items defined in the `TreesSink` trait to be implemented for the `RcDomWithLineTracking` struct. This expectation arises even when using `Deref` and `DerefMut` for method delegation because Rust's trait implementation mechanism doesn't automatically infer that methods called on the `deref` target should satisfy trait requirements.

To resolve this issue while still leveraging `Deref` for convenience, you'll need to explicitly forward trait methods to the `RcDom` instance within your `RcDomWithLineTracking` struct. However, for the `Treesink` trait, which expects you to implement a significant number of methods, manually forwarding each method can become cumbersome.

Given this, while the `Deref / DerefMut` approach can simplify access to the underlying `RcDom` methods for regular use, it doesn't exempt you from the requirement to explicitly implement the trait methods needed to conform to the `Treesink` trait.

For the specific methods mentioned in the error, you'll need to explicitly implement them in your wrapper struct and manually delegate each call to the `RcDom` instance. Here's

how you might implement a few of them to demonstrate:

rust

 Copy code

```
impl TreeSink for RcDomWithLineTracking {
    type Handle = <RcDom as TreeSink>::Handle; // Forwarding the associated
    type Output = <RcDom as TreeSink>::Output; // Forwarding the associated

    // Explicitly forward the `finish` method to `RcDom`
    fn finish(self) -> Self::Output {
        self.rc_dom.finish()
    }

    // Forward the `get_document` method to `RcDom`
    fn get_document(&mut self) -> Self::Handle {
        self.rc_dom.get_document()
    }

    // Continue with other methods similarly...
    // For `parse_error`, implement your custom logic
    fn parse_error(&mut self, msg: Cow<'static, str>) {
        let augmented_msg = format!("Line {}: {}", self.current_line, msg);
        self.rc_dom.parse_error(Cow::Owned(augmented_msg));
    }

    // Explicitly forward the `create_element` method (as an example)
    fn create_element(&mut self, name: QualName, attrs: Vec<Attribute>, flags)
        self.rc_dom.create_element(name, attrs, flags)
    }

    // Continue with all other required methods...
}
```

You will need to implement this pattern for each method listed in the error message. This approach ensures that your `RcDomWithLineTracking` struct correctly implements the `TreeSink` trait by explicitly defining or forwarding all necessary methods, while still allowing you to override specific methods like `parse_error` to add your custom behavior.

So we're back to writing delegation code for like 20 methods? I thought Rust was supposed to be a good language :(

Rust is a powerful language that offers many benefits in terms of performance, safety, and concurrency. However, like any language, it has its own set of rules and patterns that can sometimes feel verbose, especially when dealing with trait implementations that require explicit method definitions.

The requirement to implement or explicitly delegate each method when working with traits is part of Rust's design for ensuring type safety and explicit behavior. This design encourages clear and deliberate code, which is beneficial for maintaining large and complex systems. However, it can indeed introduce verbosity in scenarios like yours, where a wrapper type needs to delegate a large number of methods to an inner type.

Strategies to Reduce Boilerplate

While the need for explicit implementation cannot be completely avoided in your scenario, there are strategies to manage and organize your code to make it more manageable:

- 1. Macro for Delegation:** As mentioned earlier, although Rust doesn't provide built-in delegation, you can write a macro to automate the generation of delegation methods. This approach requires some upfront effort to write the macro but can significantly reduce boilerplate afterward. However, writing a comprehensive macro for method delegation can be complex and might not always be the most time-efficient solution, depending on your project's scope.
- 2. Code Organization:** Splitting the implementation of your wrapper struct into multiple blocks or even separate modules (if the trait and struct are large enough) can help keep the code organized. Although this doesn't reduce the amount of code, it makes it easier to navigate and maintain.
- 3. Community Crates:** There might be crates available in the Rust ecosystem that attempt to solve the problem of boilerplate code for common patterns, including delegation. For example, crates like `shrinkwraps` or `delegate` provide macros to

simplify delegation. These crates offer a way to annotate methods that should be delegated, reducing the manual work required.

- 4. Re-evaluating Requirements:** Sometimes, revisiting the design to see if the extensive implementation of a trait is truly necessary or if there could be a simpler approach to achieve the desired functionality can be beneficial. This might involve considering alternative designs or leveraging other Rust features.

Rust's Philosophy

It's important to note that Rust's emphasis on explicitness and type safety is a deliberate design choice that, while it can lead to verbosity in some cases, provides benefits in terms of performance, reliability, and predictability of code. This trade-off is generally considered worthwhile in the Rust community, especially for larger projects where the cost of bugs and unclear behavior can be high.

Conclusion

While Rust's requirement for explicit trait implementations can introduce verbosity, the language's design choices are aimed at ensuring safety and clarity. For cases like yours,